



GPS -BASED VEHICLE TRACKING AND FLEET MANAGEMENT SYSTEM

EXERCISE NO – 13

HARRISH T M	– 24MER016
KAPIL P	– 24MER022
RAMPRASATH D	– 24MER047
SARAVANAKARTHIK S R	– 24MER055
SETHUPATHI M	– 24MER056

13. GPS -BASED VEHICLE TRACKING AND FLEET MANAGEMENT SYSTEM

Project overview:

The IoT-Based GPS Vehicle Tracking System is designed to monitor the real-time location of a vehicle using a GPS module and an **ESP8266** Wi-Fi microcontroller.

The **NEO-6M GPS** module receives signals from GPS satellites and obtains the vehicle's geographical position. The **ESP8266** reads this GPS information and connects to the internet through Wi-Fi. The collected information is then uploaded to Adafruit IO, where it can be viewed remotely through a dashboard.

The system provides:

- Latitude
- Longitude
- Vehicle speed
- Number of GPS satellites
- Vehicle location on a map

The project demonstrates the integration of GPS, IoT, Wi-Fi, cloud computing and web-based monitoring.

Problem Statement:

Traditional vehicle monitoring methods may not provide convenient real-time information about the vehicle's location.

The main problems addressed by this project are:

- Difficulty in knowing the current vehicle location remotely.
- Lack of continuous GPS-based monitoring.
- Difficulty in accessing vehicle location data from different places.
- No simple cloud-based system for storing GPS information.
- Lack of an online map interface for monitoring the vehicle.

Therefore, a low-cost IoT system is required to collect GPS information and make it available remotely through the internet.

Proposed Solution:

The proposed system combines GPS and IoT technologies.

Hardware:

- ESP8266
- NEO-6M GPS module
- Wi-Fi network
- Power supply

Software:

- Arduino IDE
- Tiny GPS Plus library
- Software Serial library
- Adafruit IO Arduino library
- Adafruit IO dashboard.

The NEO-6M obtains the GPS coordinates.

The ESP8266 processes the GPS information and connects to Wi-Fi.

The ESP8266 then sends the data to Adafruit IO.

System Architecture:

The system has four main layers.

Layer 1 - (GPS Layer)

The NEO-6M receives signals from GPS satellites.

It provides:

- Latitude
- Longitude
- Speed
- Altitude
- Date/time
- Satellite information

Layer 2 – (Processing Layer)

- The ESP8266 receives GPS data through serial communication.
- The Tiny GPSPlus library is used to decode the GPS information.

Layer 3 – (Communication Layer)

- The ESP8266 connects to a Wi-Fi network.
- The processed data is transmitted to Adafruit IO using the internet.

Layer 4 – (Cloud and Monitoring Layer)

Adafruit IO receives and stores the information in feeds.

The Dashboard displays:

- Latitude
- Longitude
- Speed
- Satellites
- Map location

Circuit Table:

Because you are using Generic ESP8266 Module, use GPIO numbers rather than D1, D2, D5, etc

NEO -6M PIN	ESP8266 Pin	Function
TX	GPIO14	GPS Data - ESP8266
RX	GPIO12	ESP8266 – GPS Data
VCC	3.3V	Power
GND	GND	Ground

Circuit Design:

The circuit consists mainly of the ESP8266 and NEO-6M GPS module.

ESP8266:

The ESP8266 is the main controller.

- Receives GPS data.
- Decodes GPS information.
- Connects to Wi-Fi.
- Sends data to Adafruit IO.

NEO-6M:

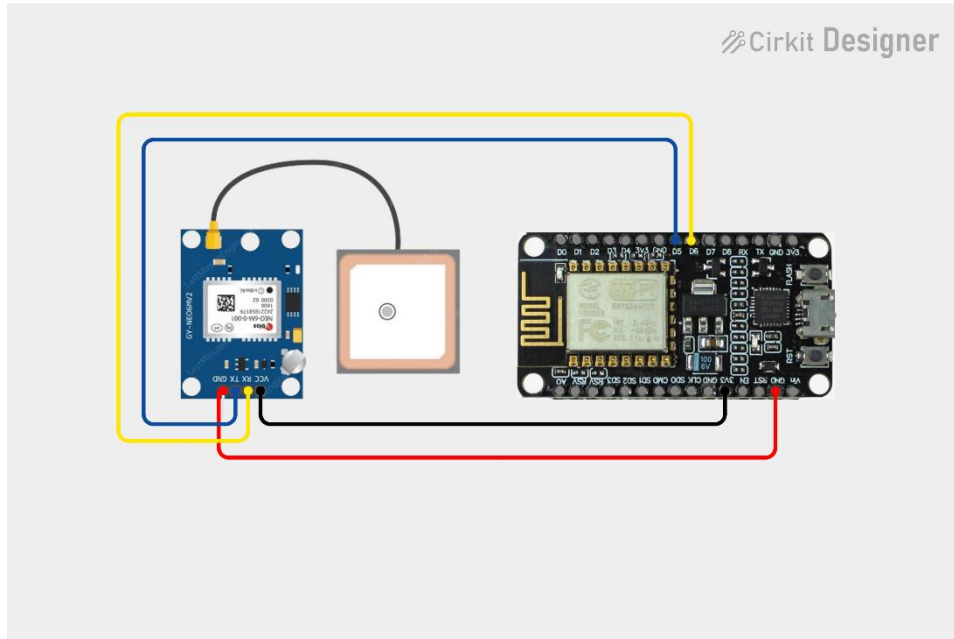
- The GPS module determines the geographical position using satellite signals.
- The GPS communicates with the ESP8266 using serial communication at 9600 baud in this project.

Power:

- The modules require a suitable regulated power supply. Connect the grounds together.

No display:

- The system does not use an OLED. All monitoring is performed through:
 - Arduino Serial Monitor
 - Adafruit IO dashboard



Algorithm:

Step-by-step algorithm

1. Start the ESP8266.
2. Initialize serial communication.
3. Initialize GPS communication at 9600 baud.
4. Connect ESP8266 to Wi-Fi.
5. Connect ESP8266 to Adafruit IO.
6. Read GPS serial data continuously.
7. Parse GPS data using Tiny GPSPlus.
8. Check whether a valid GPS location is available.
9. Read latitude.
10. Read longitude.
11. Read speed.
12. Read satellite count.
13. Send the values to their respective Adafruit IO feeds.
14. Send location information for the map.
15. Repeat continuously.

CODING:

```
#include <ESP8266WiFi.h>
#include "AdafruitIO_WiFi.h"
#include <TinyGPSPlus.h>
#include <SoftwareSerial.h>
#define WIFI_SSID "Harrish"
#define WIFI_PASS "Password"
#define IO_USERNAME "saravanakarthiks"
#define IO_KEY      "aio_idPg82y91fmNVXL0iJA71zKwH97q"
AdafruitIO_WiFi io(
  IO_USERNAME,
  IO_KEY,
  WIFI_SSID,
  WIFI_PASS
);

AdafruitIO_Feed *latitudeFeed = io.feed("latitude");
AdafruitIO_Feed *longitudeFeed = io.feed("longitude");
AdafruitIO_Feed *speedFeed    = io.feed("speed");
AdafruitIO_Feed *satelliteFeed = io.feed("satellites");
AdafruitIO_Feed *locationFeed = io.feed("gps-location");
#define GPS_RX_PIN 14
#define GPS_TX_PIN 12
SoftwareSerial gpsSerial(GPS_RX_PIN, GPS_TX_PIN);
TinyGPSPlus gps;
unsigned long lastUpload = 0;
const unsigned long uploadInterval = 10000; // 10 seconds
void setup()
{
  Serial.begin(115200);
  delay(1000);
  Serial.println();
  Serial.println();
  Serial.println("GPS VEHICLE TRACKING SYSTEM");
  Serial.println();
  Serial.println("ESP8266 + NEO-6M + Adafruit IO");
  Serial.println();
  gpsSerial.begin(9600);
  Serial.println("GPS started.");
  Serial.println("GPS Baud Rate: 9600");
  Serial.println();
  Serial.println("Connecting to Adafruit IO...");
  io.connect();
```

```

while (io.status() < AIO_CONNECTED)
{
  Serial.print(".");
  delay(500);
}

Serial.println();
Serial.println("Adafruit IO connected!");
Serial.println();
}
void loop()
{
  io.run();
  while (gpsSerial.available() > 0)
  {
    char c = gpsSerial.read();

    gps.encode(c);
  }

  if (gps.location.isUpdated())
  {
    double latitude = gps.location.lat();
    double longitude = gps.location.lng();

    double speed = gps.speed.kmph();

    int satellites = gps.satellites.value();

    Serial.println("");

    Serial.print("Latitude : ");
    Serial.println(latitude, 6);

    Serial.print("Longitude : ");
    Serial.println(longitude, 6);

    Serial.print("Speed   : ");
    Serial.print(speed, 2);
    Serial.println(" km/h");

    Serial.print("Satellites: ");
    Serial.println(satellites);

    if (millis() - lastUpload >= uploadInterval)
    {
      lastUpload = millis();
    }
  }
}

```

```
Serial.println();  
Serial.println("Uploading GPS data to Adafruit IO...");
```

```
latitudeFeed->save(latitude);
```

```
Serial.print("Latitude uploaded: ");  
Serial.println(latitude, 6);
```

```
longitudeFeed->save(longitude);
```

```
Serial.print("Longitude uploaded: ");  
Serial.println(longitude, 6);
```

```
speedFeed->save(speed);
```

```
Serial.print("Speed uploaded: ");  
Serial.print(speed, 2);  
Serial.println(" km/h");
```

```
    satelliteFeed->save(satellites);
```

```
Serial.print("Satellites uploaded: ");  
Serial.println(satellites);
```

```
double altitude = 0;
```

```
if (gps.altitude.isValid())  
{  
    altitude = gps.altitude.meters();  
}
```

```
locationFeed->save(  
    speed,  
    latitude,  
    longitude,  
    altitude  
);
```

```
Serial.println("Map location uploaded.");
```

```
Serial.println("");  
Serial.println("UPLOAD COMPLETE");  
Serial.println("");  
Serial.println();
```

```
}  
}  
  
delay(10);  
}
```

SYSTEM WORKING:

The complete working process is as follows.

Step 1 — Power ON

The ESP8266 and NEO-6M are powered.

Step 2 — GPS starts

The NEO-6M starts receiving satellite signals.

The first GPS fix can take some time, especially when starting for the first time.

Step 3 — GPS data received

The NEO-6M sends serial data to the ESP8266.

Example:

Latitude : 11.275123

Longitude : 77.603456

Speed : 25.40 km/h

Satellites: 7

Step 4 — ESP8266 processes data

The Tiny GPSPlus library extracts useful information from the GPS data.

Step 5 — Wi-Fi connection

The ESP8266 connects to the configured Wi-Fi network.

Step 6 — Adafruit IO connection

The ESP8266 connects to the user's Adafruit IO account.

Step 7 — Data upload

The ESP8266 sends:

Latitude

Longitude

Speed

Satellite count

to the corresponding feeds.

Step 8 — Dashboard

The Adafruit IO dashboard displays the received information.

For example:

LATITUDE

11.275123

LONGITUDE

77.603456

SPEED

25.4 km/h

SATELLITES

7

The location feed can also be used to display the vehicle on the map.

COM5

GPS Time : 10:21:56

GPS characters received: 176243

GPS FIX: YES

Latitude : 11.269893

Longitude : 77.603513

Speed : 0.09 km/h

Altitude : 338.20 m

Satellites: 6

Date : 13/8/2026

GPS Time : 10:21:56

GPS characters received: 176837

GPS FIX: YES

Latitude : 11.269859

Longitude : 77.603512

Speed : 0.52 km/h

Altitude : 337.50 m

Satellites: 6

Date : 13/8/2026

GPS Time : 10:21:57

GPS characters received: 177431

GPS FIX: YES

Latitude : 11.269824

Longitude : 77.603513

Speed : 3.13 km/h

Altitude : 336.80 m

Satellites: 6

Date : 13/8/2026

GPS Time : 10:21:58

GPS characters received: 178025

GPS FIX: YES

Latitude : 11.269893

Longitude : 77.603374

Speed : 1.28 km/h

Altitude : 300.40 m

Satellites: 6

Date : 13/8/2026

GPS Time : 10:21:59

GPS characters received: 178619

GPS FIX: YES

Latitude : 11.270071

Longitude : 77.603491

Speed : 1.11 km/h

Altitude : 269.60 m

Satellites: 6

Date : 13/8/2026

GPS Time : 10:22:00

GPS characters received: 179213

GPS FIX: YES

Latitude : 11.269999

Longitude : 77.603582

Speed : 0.24 km/h

Altitude : 247.10 m

Satellites: 6

Date : 13/8/2026

GPS Time : 10:22:01

☒ Autoscroll

☐ Show timestamp

adafruit

Devices

Feeds

Dashboards

Actions

Power-Ups

New Device

saravankarthiksr / Dashboards / GPS\

New Block

Edit Layout

LATITUDE

11.26977

LONGITUDE

77.603338

SPEED

0.000000

GPS STATUS

6

LOCATION

Mechanical and Mechatronics block

3

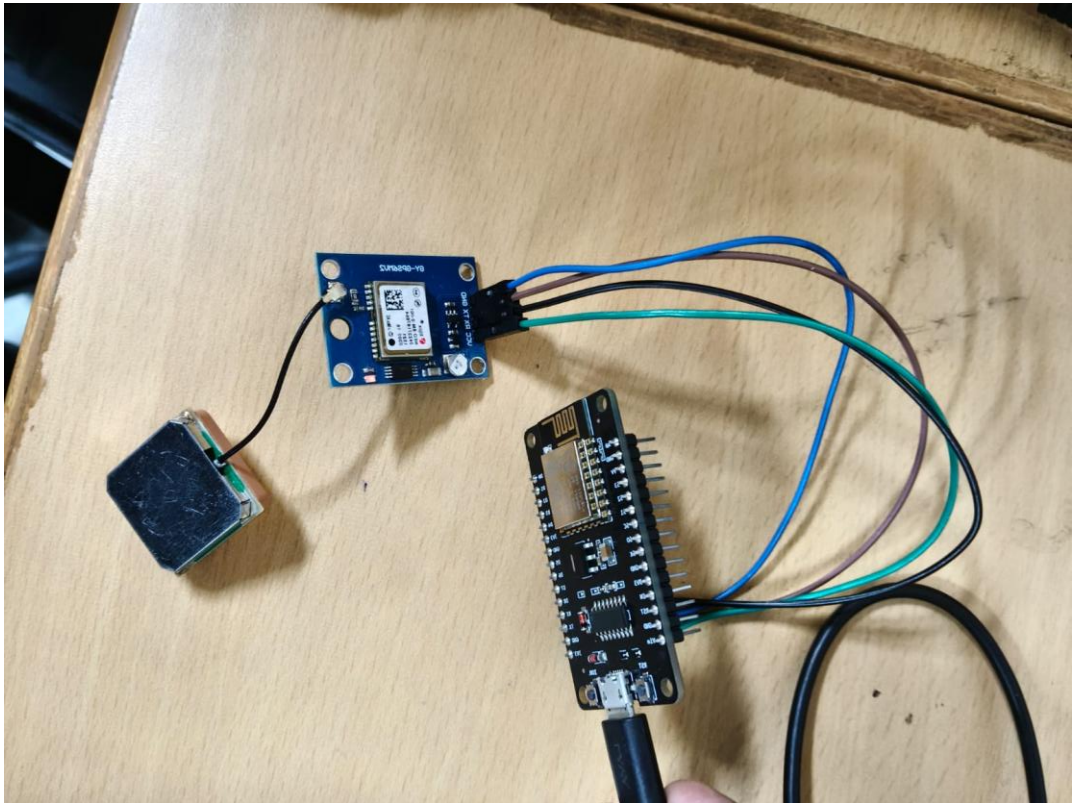
gps-location

Leaflet | © OpenStreetMap

Bill of Materials:

COMPONENT	QUANTITY
ESP8266	1
NEO-6M GPS Module	1
LI-ion Battery	1
Jumper Wires	Required

Prototype Implementation:



Results & Performance Analysis:

The project can be evaluated using the following parameters.

1. GPS location

The system successfully obtains:

Latitude

Longitude

when the GPS has a valid satellite fix.

2. Speed

The NEO-6M provides speed information, which is uploaded to the speed feed.

Example:

Speed = 32.5 km/h

3. Satellite count

The satellites feed indicates the number of satellites reported by the GPS.

Example:

Satellites = 7

A higher number generally indicates better GPS reception, although satellite count alone does not guarantee a particular accuracy.

4. Cloud connectivity

When Wi-Fi and internet access are available, the ESP8266 sends the data to Adafruit IO.

5. Map tracking

The location feed can be used to display the vehicle position on the Adafruit IO Map.

6. Upload interval

The proposed code uploads information approximately every 10 seconds.

7. Indoor performance

A closed room can reduce GPS signal reception. In this situation:

Wi-Fi → May work normally

ESP8266 → Works normally

Adafruit IO → Can work normally

GPS → May not obtain a valid fix

Therefore, the GPS antenna should preferably have a clear view of the sky.